

Set-1

Create following tables with primary and foreign keys and solve the queries given below. Primary keys are underlined

Person (driver_id, name, address)

Car (license, model, year)

Accident (report_no, date_acc, location)

Owns (driver_id, license)

Participated (driver_id, model, report_no, damage_amount)

- 1) Insert 5 records into each table
- 2) Display the Schema of each table.
- 3) Display all records from the person table
- 4) Set a **Foreign key** constraint from Participated.report_no to Accident.report_no.
- 5) Add check constraints on Damage_amount that amt should not be negative.
- 6) Write a query to make the name column in the Person table **NOT NULL**.
- 7) Rename the Car table to Vehicle.
- 8) Create a new table named new_person from existing person table and drop the table new_person.

```
CREATE TABLE Person(  
driver_id NUMBER PRIMARY KEY,  
name VARCHAR2(50),  
address VARCHAR2(100)  
);
```

```
CREATE TABLE Car(  
license VARCHAR2(20) PRIMARY KEY,  
model VARCHAR2(30),  
year NUMBER  
);
```

```
CREATE TABLE Accident(  
report_no NUMBER PRIMARY KEY,  
date_acc DATE,  
location VARCHAR2(50)  
);
```

```
CREATE TABLE Owns(  
driver_id NUMBER,  
license VARCHAR2(20),  
PRIMARY KEY(driver_id,license),  
FOREIGN KEY(driver_id) REFERENCES Person(driver_id),  
FOREIGN KEY(license) REFERENCES Car(license)  
);
```

```
CREATE TABLE Participated(  
driver_id NUMBER,  
model VARCHAR2(30),  
report_no NUMBER,  
damage_amount NUMBER  
);
```

```
INSERT INTO Person VALUES(1,'Amit','Pune');
```

```
INSERT INTO Person VALUES(2,'Neha','Mumbai');
INSERT INTO Person VALUES(3,'Raj','Nashik');
INSERT INTO Person VALUES(4,'Pooja','Delhi');
INSERT INTO Person VALUES(5,'Karan','Chennai');
```

```
INSERT INTO Car VALUES('MH12AB1234','Honda',2020);
INSERT INTO Car VALUES('MH14XY5678','Hyundai',2019);
INSERT INTO Car VALUES('MH15PQ1111','Toyota',2021);
INSERT INTO Car VALUES('MH16LM2222','Suzuki',2022);
INSERT INTO Car VALUES('MH17GH3333','Kia',2018);
```

```
INSERT INTO Accident VALUES(101,TO_DATE('12-01-2024','DD-MM-YYYY'),'Pune');
INSERT INTO Accident VALUES(102,TO_DATE('15-02-2024','DD-MM-YYYY'),'Mumbai');
INSERT INTO Accident VALUES(103,TO_DATE('18-03-2024','DD-MM-YYYY'),'Nashik');
INSERT INTO Accident VALUES(104,TO_DATE('20-04-2024','DD-MM-YYYY'),'Delhi');
INSERT INTO Accident VALUES(105,TO_DATE('25-05-2024','DD-MM-YYYY'),'Chennai');
```

```
DESC Person;
DESC Car;
DESC Accident;
DESC Owns;
DESC Participated;
```

```
SELECT * FROM Person;
```

```
ALTER TABLE Participated
ADD CONSTRAINT fk_report
FOREIGN KEY(report_no)
REFERENCES Accident(report_no);
```

```
ALTER TABLE Participated
ADD CONSTRAINT chk_damage
CHECK(damage_amount >= 0);
```

```
ALTER TABLE Person
MODIFY name VARCHAR2(50) NOT NULL;
```

```
RENAME Car TO Vehicle;
```

```
CREATE TABLE new_person AS
SELECT * FROM Person;
```

```
DROP TABLE new_person;
```

Set 2

Create Customer and Account table and add rows shown below

Customer Table

C_Id	Cname	City
1	John	Nashik
2	Seema	Aurangabad
3	Amita	Nagar
4	Rakesh	Pune
5	Samata	Nashik
6	Ankita	Chandwad
7	Bhavika	Pune
8	Deepa	Mumbai
9	Nitin	Nagpur
10	Pooja	Pune

Account Table

C_Id	Acc_Type	Amount
1	Current	5000
2	Saving	20000
3	Saving	70000
4	Saving	50000
6	Current	35000
7	Loan	30000
8	Saving	50000
9	Saving	90000
10	Loan	8000
11	Current	45000

- 1) Show the cname, Acc_Type, amount information of customer who is having an saving account.
2. Display the data using Natural, left and right join.
3. Display the information of customers living in the same city as of 'pooja'.
4. Display the information of account, having less amount than average amount throughout the bank.
5. Display the C_id having maximum amount in account.
6. Display the amount and acc_type of those customers whose amount is the minimum amount of that Acc_type.
7. Display the amount of those accounts whose amount is higher than amount of any saving account amount.

```
CREATE TABLE Customer(  
    c_id NUMBER PRIMARY KEY,  
    cname VARCHAR2(30),  
    city VARCHAR2(30)  
);
```

```
CREATE TABLE Account(  
    c_id NUMBER,  
    acc_type VARCHAR2(20),  
    amount NUMBER  
);
```

```
INSERT INTO Customer VALUES(1,'John','Nashik');  
INSERT INTO Customer VALUES(2,'Seema','Aurangabad');  
INSERT INTO Customer VALUES(3,'Amita','Nagar');  
INSERT INTO Customer VALUES(4,'Rakesh','Pune');  
INSERT INTO Customer VALUES(5,'Samata','Nashik');  
INSERT INTO Customer VALUES(6,'Ankita','Chandwad');  
INSERT INTO Customer VALUES(7,'Bhavika','Pune');  
INSERT INTO Customer VALUES(8,'Deepa','Mumbai');  
INSERT INTO Customer VALUES(9,'Nitin','Nagpur');  
INSERT INTO Customer VALUES(10,'Pooja','Pune');
```

```
INSERT INTO Account VALUES(1,'Current',5000);  
INSERT INTO Account VALUES(2,'Saving',20000);  
INSERT INTO Account VALUES(3,'Saving',70000);  
INSERT INTO Account VALUES(4,'Saving',50000);
```

```
INSERT INTO Account VALUES(6,'Current',35000);
INSERT INTO Account VALUES(7,'Loan',30000);
INSERT INTO Account VALUES(8,'Saving',50000);
INSERT INTO Account VALUES(9,'Saving',90000);
INSERT INTO Account VALUES(10,'Loan',8000);
INSERT INTO Account VALUES(11,'Current',45000);
```

```
SELECT cname,acc_type,amount
FROM Customer NATURAL JOIN Account
WHERE acc_type='Saving';
```

Natural Join

```
SELECT *
FROM Customer NATURAL JOIN Account;
```

Left Outer Join

```
SELECT *
FROM Customer
LEFT JOIN Account
USING(c_id);
```

Right Outer Join

```
SELECT *
FROM Customer
RIGHT JOIN Account
USING(c_id);
```

Customers living in same city as Pooja

```
SELECT *
FROM Customer
WHERE city=(
SELECT city
FROM Customer
WHERE cname='Pooja'
);
```

Accounts having amount less than average amount

```
SELECT *
FROM Account
WHERE amount<(
SELECT AVG(amount)
FROM Account
);
```

Customer having maximum account amount

```
SELECT c_id
FROM Account
WHERE amount=(
SELECT MAX(amount)
FROM Account
);
```

Minimum amount according to account type

```
SELECT amount,acc_type
FROM Account a
WHERE amount=(
SELECT MIN(amount)
FROM Account
```

```
WHERE acc_type=a.acc_type
);
```

Amount greater than ANY saving account amount

```
SELECT amount
FROM Account
WHERE amount>ANY(
SELECT amount
FROM Account
WHERE acc_type='Saving'
);
```

Set 3

Create Employee and Department table by identifying Primary key and foreign keys and solve the following queries

Employee (emp_id, emp_name, city, salary, department)

Department (dept_id, dept_name, location)

1. Display schema of each table.in each table.
2. Insert 10 records in each table
3. Insert a new department into the Department table.
4. Insert an employee record without specifying the salary column.
5. Update the salary of all employees by adding 20% increment.
6. Display employee name and annual salary of employees whose names having the second letter e.
7. Update the city to 'Pune' for employees working in the 'IT' department.

```
CREATE TABLE Department(
dept_id NUMBER PRIMARY KEY,
dept_name VARCHAR2(30),
location VARCHAR2(30)
);

CREATE TABLE Employee(
emp_id NUMBER PRIMARY KEY,
emp_name VARCHAR2(30),
city VARCHAR2(30),
salary NUMBER,
department NUMBER,
FOREIGN KEY(department)
REFERENCES Department(dept_id)
);
```

```
INSERT INTO Department VALUES(1,'IT','Pune');
INSERT INTO Department VALUES(2,'HR','Mumbai');
INSERT INTO Department VALUES(3,'Sales','Delhi');
```

```
INSERT INTO Employee VALUES(101,'Neeraj','Pune',50000,1);
INSERT INTO Employee VALUES(102,'Amit','Mumbai',45000,2);
INSERT INTO Employee VALUES(103,'Deepa','Delhi',60000,3);
```

New dept :

```
INSERT INTO Department
VALUES(4,'Marketing','Nagpur');
```

```
UPDATE Employee
SET salary=salary+(salary*0.20);
```

```
INSERT INTO Employee(emp_id,
emp_name,city,department)VALUES(104,'Kunal','Pune',1);
```

```
SELECT emp_name,  
salary*12 AS annual_salary  
FROM Employee  
WHERE emp_name LIKE '_e%';
```

```
UPDATE Employee  
SET city='Pune'  
WHERE department=(  
SELECT dept_id  
FROM Department  
WHERE dept_name='IT'  
);
```

Set 4

Create Employee and Department table by identifying Primary key and foreign keys

Employee (emp_id, emp_name, city, salary, department)

Department (dept_id, dept_name, location)

1. Display schema of each table.
2. Insert 10 records in each table
3. Update the city to 'Pune' for employees working in the 'IT' department.
4. Delete employee records whose salary is greater than 100000.
5. Display employee name and salary after deducting 5% tax.
6. Display employees who do not live in 'Mumbai' and whose names start with a.
7. Display employees who belong to 'IT' department and have salary greater than 50000.

```
DESC Department;  
DESC Employee;
```

```
UPDATE Employee  
SET city='Pune'  
WHERE department=(  
SELECT dept_id  
FROM Department  
WHERE dept_name='IT'  
);
```

```
DELETE FROM Employee  
WHERE salary>100000;
```

```
SELECT emp_name,  
salary-(salary*0.05) AS salary_after_tax  
FROM Employee;
```

```
SELECT *  
FROM Employee  
WHERE city<>'Mumbai'  
AND emp_name LIKE 'A%';
```

```
SELECT *  
FROM Employee  
WHERE department=(  
SELECT dept_id  
FROM Department  
WHERE dept_name='IT'  
)  
AND salary>50000;
```

Set 5

1. Create a new user named with Class_div_rollno.
2. Grant SELECT permission on the Employee table to a user named Class_div_rollno
3. Grant INSERT and UPDATE permissions on the Employee table to Class_div_rollno
4. Revoke UPDATE permission from user1.
5. Create a role named manager_role.
6. Grant SELECT and DELETE privileges on Employee table to manager_role.
7. Revoke the role manager_role from user2

```
CREATE USER C##class_div_rollno  
IDENTIFIED BY password123;
```

```
GRANT SELECT  
  ON Employee  
  TO C##class_div_rollno;
```

Grant INSERT and UPDATE permissions

```
GRANT INSERT,UPDATE  
  ON Employee  
  TO C##class_div_rollno;
```

Revoke UPDATE permission

```
REVOKE UPDATE  
  ON Employee  
  FROM C##class_div_rollno;
```

Create Role

```
CREATE ROLE manager_role;
```

Grant permissions to role

```
GRANT SELECT,DELETE  
  ON Employee  
  TO manager_role;
```

Assign role to user

```
GRANT manager_role  
  TO C##class_div_rollno;
```

Remove role from user

```
REVOKE manager_role  
  FROM C##class_div_rollno;
```

Set 6

Create the tables below and solve the following queries.

Employees

Name	Null?	Type

EMPLOYEE_ID	NOT NULL	NUMBER(6)
FIRST_NAME		VARCHAR2(20)
LAST_NAME	NOT NULL	VARCHAR2(25)
EMAIL	NOT NULL	VARCHAR2(25)
PHONE_NUMBER		VARCHAR2(20)
HIRE_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2(10)
SALARY		NUMBER(8,2)
COMMISSION_PCT		NUMBER(2,2)
MANAGER_ID		NUMBER(6)
DEPARTMENT_ID		NUMBER(4)

Departments

Name	Null?	Type

DEPARTMENT_ID	NOT NULL	NUMBER(4)
DEPARTMENT_NAME	NOT NULL	VARCHAR2(30)
MANAGER_ID		NUMBER(6)
LOCATION_ID		NUMBER(4)

1. Create EmpTest from Emp table by copying structure and data.
2. Add a new row into the EmpTest table for Empno, Ename, Sal columns. In Ename should have the Current User.
3. Update EmpTest by increasing the Salary of Vaishali by 15%. Confirm your changes.
4. Increase all the employees salary by 10% in EmpTest table who are working in PUNE
5. Delete all the Comm data from EmpTest Table.
6. Delete all the employees in deptno 10 and do not issue a Commit.
7. Issue a Rollback and reflect changes.

```
CREATE TABLE Departments(
```

```
    DEPARTMENT_ID NUMBER(4) NOT NULL,  
    DEPARTMENT_NAME VARCHAR2(30) NOT NULL,  
    MANAGER_ID NUMBER(6),  
    LOCATION_ID NUMBER(4),  
    PRIMARY KEY(DEPARTMENT_ID)  
);
```

```
CREATE TABLE Employees(
```

```
    EMPLOYEE_ID NUMBER(6) NOT NULL,  
    FIRST_NAME VARCHAR2(20),  
    LAST_NAME VARCHAR2(25) NOT NULL,  
    EMAIL VARCHAR2(25) NOT NULL,  
    PHONE_NUMBER VARCHAR2(20),  
    HIRE_DATE DATE NOT NULL,
```



```

JOB_ID VARCHAR2(10) NOT NULL,
SALARY NUMBER(8,2),
COMMISSION_PCT NUMBER(2,2),
MANAGER_ID NUMBER(6),
DEPARTMENT_ID NUMBER(4),
PRIMARY KEY(EMPLOYEE_ID),
FOREIGN KEY(DEPARTMENT_ID)
REFERENCES Departments(DEPARTMENT_ID)
);

```

Department Table

```

INSERT INTO Department
VALUES(1,'IT','Pune');
INSERT INTO Department
VALUES(2,'HR','Mumbai');
INSERT INTO Department
VALUES(3,'Sales','Delhi');

```

Employees Table

```

INSERT INTO Employees
VALUES(101,'Vaishali','Patil',45000,1);
INSERT INTO Employees
VALUES(102,'Amit','Sharma',30000,2);
INSERT INTO Employees
VALUES(103,'Neha','Joshi',25000,3);
INSERT INTO Employees
VALUES(104,'Raj','Mehta',18000,1);
INSERT INTO Employees
VALUES(105,'Pooja','Kale',50000,2);

```

```

CREATE TABLE EmpTest AS
SELECT *
FROM Employees;

```

```

INSERT INTO EmpTest(
Empno,
Ename,
Sal
)
VALUES(
101,
USER,
25000
);

```

```

UPDATE EmpTest
SET salary=salary+(salary*0.15)
WHERE first_name='Vaishali';

```

```

COMMIT;

```

```

UPDATE EmpTest
SET Sal=Sal+(Sal*0.10)
WHERE City='PUNE';

```

```
UPDATE EmpTest
SET Comm=NULL;
```

6. Delete all employees in deptno 10 and do not issue COMMIT

```
DELETE FROM EmpTest
WHERE deptno=10;
```

7. Issue ROLLBACK and reflect changes

```
ROLLBACK;
```

```
SELECT * FROM EmpTest;
```

Set 7

Create the table below and solve the following queries.

Employees

Name	Null?	Type

EMPLOYEE_ID	NOT NULL	NUMBER(6)
FIRST_NAME		VARCHAR2(20)
LAST_NAME	NOT NULL	VARCHAR2(25)
EMAIL	NOT NULL	VARCHAR2(25)
PHONE_NUMBER		VARCHAR2(20)
HIRE_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2(10)
SALARY		NUMBER(8,2)
COMMISSION_PCT		NUMBER(2,2)
MANAGER_ID		NUMBER(6)
DEPARTMENT_ID		NUMBER(4)

1. Start a transaction and insert a new employee record. Commit the transaction.
2. Start a transaction, update employee salary, and rollback the transaction.
3. Start a transaction, create a savepoint named SP1, update salary, and rollback to SP1.
4. Demonstrate the use of multiple savepoints within a single transaction.
5. Start a transaction, delete an employee record, and commit the changes.

```
CREATE TABLE Employees(
EMPLOYEE_ID NUMBER(6) NOT NULL,
FIRST_NAME VARCHAR2(20),
LAST_NAME VARCHAR2(25) NOT NULL,
EMAIL VARCHAR2(25) NOT NULL,
PHONE_NUMBER VARCHAR2(20),
HIRE_DATE DATE NOT NULL,
JOB_ID VARCHAR2(10) NOT NULL,
SALARY NUMBER(8,2),
COMMISSION_PCT NUMBER(2,2),
MANAGER_ID NUMBER(6),
DEPARTMENT_ID NUMBER(4),
PRIMARY KEY(EMPLOYEE_ID)
);
```

```
INSERT INTO Employees VALUES(101,'Amit','Sharma','amit@gmail.com','9876543210',TO_DATE('10-01-2022','DD-MM-YYYY'),'IT_PROG',45000,0.10,201,10);
INSERT INTO Employees VALUES(102,'Neha','Patil','neha@gmail.com','9988776655',TO_DATE('15-03-2021','DD-MM-YYYY'),'HR_MGR',35000,0.05,202,20);
INSERT INTO Employees VALUES(103,'Raj','Joshi','raj@gmail.com','9123456780',TO_DATE('20-07-2020','DD-MM-YYYY'),'SA_REP',50000,0.08,203,30);
```

1. Start a transaction and insert a new employee record. Commit the transaction.

```
INSERT INTO Employees VALUES(104,'Pooja','Kale','pooja@gmail.com','9871234567',TO_DATE('01-02-2023','DD-MM-YYYY'),'IT_PROG',40000,0.07,204,10);
COMMIT;
```

```
UPDATE Employees
SET SALARY=SALARY+(SALARY*0.15)
WHERE EMPLOYEE_ID=101;
ROLLBACK;
```

```
SAVEPOINT SP1;
UPDATE Employees
SET SALARY=SALARY+2000
WHERE EMPLOYEE_ID=102;
ROLLBACK TO SP1;
```

```
SAVEPOINT SP1;
UPDATE Employees
SET SALARY=SALARY+1000
WHERE EMPLOYEE_ID=101;
SAVEPOINT SP2;
UPDATE Employees
SET SALARY=SALARY+2000
WHERE EMPLOYEE_ID=102;
ROLLBACK TO SP2;
```

```
DELETE FROM Employees
WHERE EMPLOYEE_ID=103;
COMMIT;
```

Set 8

Create the employee table with all constraints.

Name	Null?	Type

EMPLOYEE_ID	NOT NULL	NUMBER(6)
FIRST_NAME		VARCHAR2(20)
LAST_NAME	NOT NULL	VARCHAR2(25)
EMAIL	NOT NULL	VARCHAR2(25)
PHONE_NUMBER		VARCHAR2(20)
HIRE_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2(10)
SALARY		NUMBER(8,2)
COMMISSION_PCT		NUMBER(2,2)
MANAGER_ID		NUMBER(6)
DEPARTMENT_ID		NUMBER(4)

1. Find the highest, lowest, sum and average salary of all employees. Label the columns.
2. write a query to display the number of people with the same job.
3. Determine the number of managers without listing them. Label the column number of Managers
4. Find the difference between the highest and lowest salaries. Label the column DIFFERENCE.
5. Create a report to display the manager number and the salary of the lowest-paid employee for that manager.

```
CREATE TABLE Employees(  
  EMPLOYEE_ID NUMBER(6) NOT NULL,  
  FIRST_NAME VARCHAR2(20),  
  LAST_NAME VARCHAR2(25) NOT NULL,  
  EMAIL VARCHAR2(25) NOT NULL,  
  PHONE_NUMBER VARCHAR2(20),  
  HIRE_DATE DATE NOT NULL,  
  JOB_ID VARCHAR2(10) NOT NULL,  
  SALARY NUMBER(8,2),  
  COMMISSION_PCT NUMBER(2,2),  
  MANAGER_ID NUMBER(6),  
  DEPARTMENT_ID NUMBER(4),  
  PRIMARY KEY(EMPLOYEE_ID)  
);
```

Insert Queries

```
INSERT INTO Employees VALUES(101,'Amit','Sharma','amit@gmail.com','9876543210',TO_DATE('10-01-2022','DD-MM-YYYY'),'IT_PROG',45000,0.10,201,10);  
INSERT INTO Employees VALUES(102,'Neha','Patil','neha@gmail.com','9988776655',TO_DATE('15-03-2021','DD-MM-YYYY'),'HR_MGR',35000,0.05,202,20);  
INSERT INTO Employees VALUES(103,'Raj','Joshi','raj@gmail.com','9123456780',TO_DATE('20-07-2020','DD-MM-YYYY'),'SA_REP',50000,0.08,203,30);  
INSERT INTO Employees VALUES(104,'Pooja','Kale','pooja@gmail.com','9871234567',TO_DATE('01-02-2023','DD-MM-YYYY'),'IT_PROG',40000,0.07,201,10);  
INSERT INTO Employees VALUES(105,'Kunal','Mehta','kunal@gmail.com','9988123456',TO_DATE('11-11-2020','DD-MM-YYYY'),'SA_REP',30000,0.06,203,30);
```

1. Find highest, lowest, sum and average salary

```
SELECT MAX(SALARY) AS HIGHEST_SALARY,  
       MIN(SALARY) AS LOWEST_SALARY,  
       SUM(SALARY) AS TOTAL_SALARY,  
       AVG(SALARY) AS AVERAGE_SALARY  
FROM Employees;
```

2. Display number of people with same job

```
SELECT JOB_ID,  
       COUNT(*) AS TOTAL_EMPLOYEES  
FROM Employees  
GROUP BY JOB_ID;
```

3. Determine number of managers

```
SELECT COUNT(DISTINCT MANAGER_ID)  
       AS NUMBER_OF_MANAGERS  
FROM Employees;
```

4. Find difference between highest and lowest salary

```
SELECT MAX(SALARY)-MIN(SALARY)  
       AS DIFFERENCE  
FROM Employees;
```

5. Display manager number and salary of lowest-paid employee

```
SELECT MANAGER_ID,  
       MIN(SALARY) AS LOWEST_SALARY  
FROM Employees  
GROUP BY MANAGER_ID;
```

Set 9

Create the employee table with all constraints and solve the following queries. -

Employee (emp_id, emp_name, department, city, salary)

1. Display department-wise total salary of employees.
2. Display department-wise average salary.
3. City-wise employee count greater than 3
4. Department-wise maximum salary above 60,000
5. Display department and city-wise total salary.
6. Display departments having more than one employee.
7. Display city-wise employee count in descending order.

```
CREATE TABLE Employee(  
emp_id NUMBER PRIMARY KEY,  
emp_name VARCHAR2(30) NOT NULL,  
department VARCHAR2(30),  
city VARCHAR2(30),  
salary NUMBER(8,2)  
);
```

```
INSERT INTO Employee VALUES(101,'Amit','IT','Pune',65000);  
INSERT INTO Employee VALUES(102,'Neha','HR','Mumbai',45000);  
INSERT INTO Employee VALUES(103,'Raj','Sales','Pune',70000);  
INSERT INTO Employee VALUES(104,'Pooja','IT','Delhi',55000);  
INSERT INTO Employee VALUES(105,'Kunal','HR','Mumbai',48000);  
INSERT INTO Employee VALUES(106,'Sneha','IT','Pune',72000);  
INSERT INTO Employee VALUES(107,'Rohit','Sales','Delhi',62000);
```

Queries

1. Display department-wise total salary of employees

```
SELECT department,  
SUM(salary) AS total_salary  
FROM Employee  
GROUP BY department;
```

2. Display department-wise average salary

```
SELECT department,  
AVG(salary) AS average_salary  
FROM Employee  
GROUP BY department;
```

3. City-wise employee count greater than 3

```
SELECT city,  
COUNT(*) AS employee_count  
FROM Employee  
GROUP BY city  
HAVING COUNT(*)>3;
```

4. Department-wise maximum salary above 60000

```
SELECT department,  
MAX(salary) AS maximum_salary  
FROM Employee  
GROUP BY department  
HAVING MAX(salary)>60000;
```

5. Display department and city-wise total salary

```
SELECT department,  
city,  
SUM(salary) AS total_salary  
FROM Employee  
GROUP BY department,city;
```

6. Display departments having more than one employee

```
SELECT department,  
COUNT(*) AS total_employees  
FROM Employee  
GROUP BY department  
HAVING COUNT(*)>1;
```

7. Display city-wise employee count in descending order

```
SELECT city,  
COUNT(*) AS employee_count  
FROM Employee  
GROUP BY city  
ORDER BY employee_count DESC;
```

Set 10

Create the employee table with all constraints and solve the following queries. -

Employee (emp_id, emp_name, department, city, salary)

1. Department-wise total salary greater than 2,00,000
2. Department-wise average salary more than 40,000
3. Display city-wise number of employees.
4. Display department-wise maximum salary.
5. Display city-wise average salary.
6. Display department-wise employee count where the number of employees is more than 3.
7. Display city-wise employee count in ascending order.

```
CREATE TABLE Employee(  
emp_id NUMBER PRIMARY KEY,  
emp_name VARCHAR2(30) NOT NULL,  
department VARCHAR2(30),  
city VARCHAR2(30),  
salary NUMBER(8,2)  
);
```

Insert Queries

```
INSERT INTO Employee VALUES(101,'Amit','IT','Pune',65000);  
INSERT INTO Employee VALUES(102,'Neha','HR','Mumbai',45000);  
INSERT INTO Employee VALUES(103,'Raj','Sales','Pune',70000);  
INSERT INTO Employee VALUES(104,'Pooja','IT','Delhi',55000);  
INSERT INTO Employee VALUES(105,'Kunal','HR','Mumbai',48000);  
INSERT INTO Employee VALUES(106,'Sneha','IT','Pune',72000);  
INSERT INTO Employee VALUES(107,'Rohit','Sales','Delhi',62000);  
INSERT INTO Employee VALUES(108,'Deepa','IT','Pune',58000);
```

Queries

1. Department-wise total salary greater than 200000

```
SELECT department,  
SUM(salary) AS total_salary  
FROM Employee  
GROUP BY department  
HAVING SUM(salary)>200000;
```

2. Department-wise average salary more than 40000

```
SELECT department,  
AVG(salary) AS average_salary  
FROM Employee  
GROUP BY department  
HAVING AVG(salary)>40000;
```

3. Display city-wise number of employees

```
SELECT city,  
COUNT(*) AS employee_count  
FROM Employee  
GROUP BY city;
```

4. Display department-wise maximum salary

```
SELECT department,
```

```
MAX(salary) AS maximum_salary
FROM Employee
GROUP BY department;
```

5. Display city-wise average salary

```
SELECT city,
AVG(salary) AS average_salary
FROM Employee
GROUP BY city;
```

6. Display department-wise employee count where employees are more than 3

```
SELECT department,
COUNT(*) AS employee_count
FROM Employee
GROUP BY department
HAVING COUNT(*)>3;
```

7. Display city-wise employee count in ascending order

```
SELECT city,
COUNT(*) AS employee_count
FROM Employee
GROUP BY city
ORDER BY employee_count ASC;
```

Set 11

1. Create a table dept1 as below with following columns:
DEPTNO integer pk, dname varchar2(30) not null, loc varchar2(30) not null
2. Add column mgr as number in the dept table.
3. Create table empl with following columns
empno integer ,ename varchar2(20) ,sal number(10,2) ,deptno number
4. Add constraints on empl table
Empno-primary key
Ename-not null
Check sal>10000
Deptno as foreign key
5. Create table dept11,empl11 from dept1,empl
6. Add a new column address varchar2(30) in the empl table
7. Rename sal column in empl table to salary
8. Display constraints names and verify the names for the empl

```
CREATE TABLE dept1 (
  Deptno NUMBER(5) PRIMARY KEY,
  dname VARCHAR2(30) NOT NULL,
  loc VARCHAR2(30) NOT NULL
);
```

```
ALTER TABLE dept1
ADD mgr NUMBER;
```

```
CREATE TABLE empl (
  empno INTEGER,
```



```
    ename VARCHAR2(20),  
    sal NUMBER(10,2),  
    deptno NUMBER  
);
```

```
ALTER TABLE empl  
ADD CONSTRAINT pk_empno PRIMARY KEY (empno);
```

```
ALTER TABLE empl  
MODIFY ename VARCHAR2(20) NOT NULL;
```

```
ALTER TABLE empl  
ADD CONSTRAINT chk_sal CHECK (sal > 10000);
```

```
ALTER TABLE empl  
ADD CONSTRAINT fk_deptno FOREIGN KEY (deptno) REFERENCES dept1(deptno);
```

```
CREATE TABLE dept11 AS SELECT * FROM dept1;
```

```
CREATE TABLE emp11 AS SELECT * FROM empl;
```

```
ALTER TABLE empl  
ADD address VARCHAR2(30);
```

```
ALTER TABLE empl  
RENAME COLUMN sal TO salary;
```

```
SELECT constraint_name, constraint_type  
FROM user_constraints  
WHERE table_name = 'EMPL';
```

```
SELECT constraint_name, column_name  
FROM user_cons_columns  
WHERE table_name = 'EMPL';
```

Set 12

Create the tables below and solve the following queries.

Employees

Name	Null?	Type

EMPLOYEE_ID	NOT NULL	NUMBER(6)
FIRST_NAME		VARCHAR2(20)
LAST_NAME	NOT NULL	VARCHAR2(25)
EMAIL	NOT NULL	VARCHAR2(25)
PHONE_NUMBER		VARCHAR2(20)
HIRE_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2(10)
SALARY		NUMBER(8,2)
COMMISSION_PCT		NUMBER(2,2)
MANAGER_ID		NUMBER(6)
DEPARTMENT_ID		NUMBER(4)

Departments

Name	Null?	Type

DEPARTMENT_ID	NOT NULL	NUMBER(4)
DEPARTMENT_NAME	NOT NULL	VARCHAR2(30)
MANAGER_ID		NUMBER(6)
LOCATION_ID		NUMBER(4)

1. Write a query for the HR department to produce the addresses of all the departments. Use the EMP and DEPT tables. Show the EMPNO, ENAME,SAL,DNAME and LOC in the output. Use a NATURAL JOIN to produce the results.
2. The HR department needs a report of all employees. Write a query to display the JOB, MGR, SAL,COMM,DNAME of employees whose JOB is SALESMAN.
3. The HR department needs a report of employees in LOC DALLAS. Display the ENAME,job, DEPTNO and DNAME for all employees who work in DALLAS.
4. Modify the previous Query to diaplay all employees including King,who has no manager.Order the results by the employee number.

```
CREATE TABLE employees (  
  employee_id NUMBER(6),  
  first_name VARCHAR2(20) NOT NULL,  
  last_name VARCHAR2(25) NOT NULL,  
  email VARCHAR2(25) NOT NULL,  
  phone_number VARCHAR2(20),  
  hire_date DATE NOT NULL,  
  job_id VARCHAR2(10) NOT NULL,  
  salary NUMBER(8,2),  
  commission_pct NUMBER(2,2),  
  manager_id NUMBER(6),  
  department_id NUMBER(4)
```

```
);  
CREATE TABLE departments (  
    department_id NUMBER(4) NOT NULL,  
    department_name VARCHAR2(30) NOT NULL,  
    manager_id NUMBER(6),  
    location_id NUMBER(4)  
);
```

1. Natural Join (EMP + DEPT)

```
SELECT employee_id, first_name || ' ' || last_name AS ename,  
       salary, department_name, location_id  
FROM employees  
NATURAL JOIN departments;
```

2. JOB = SALESMAN report

```
SELECT job_id, manager_id, salary, commission_pct, department_name  
FROM employees e, departments d  
WHERE e.department_id = d.department_id  
AND job_id = 'SALESMAN';
```

3. Employees working in DALLAS

```
SELECT first_name || ' ' || last_name AS ename,  
       job_id, e.department_id, department_name  
FROM employees e, departments d  
WHERE e.department_id = d.department_id  
AND location_id = 'DALLAS';
```

4. Include all employees (including King) ordered by employee_id

```
SELECT first_name || ' ' || last_name AS ename,  
       job_id, manager_id, salary, commission_pct,  
       department_name  
FROM employees e  
LEFT JOIN departments d  
ON e.department_id = d.department_id  
ORDER BY employee_id;
```

Insert sample data (important for query to work)

```
e.salary, s.grade
FROM employees e, departments d, salgrade s
WHERE e.department_id = d.department_id
AND e.salary BETWEEN s.losal AND s.hisal;
```

2. Salary > 2000 (ON clause)

```
SELECT e.last_name ename, e.salary, e.manager_id, d.department_name
FROM employees e JOIN departments d
ON e.department_id = d.department_id
WHERE e.salary > 2000;
```

3. LEFT OUTER JOIN

```
SELECT e.employee_id, e.last_name ename, e.job_id, e.department_id,
       d.department_name, d.location_id
FROM employees e LEFT JOIN departments d
ON e.department_id = d.department_id;
```

4. RIGHT OUTER JOIN

```
SELECT e.last_name ename, d.department_name
FROM employees e RIGHT JOIN departments d
ON e.department_id = d.department_id;
```

5. FULL OUTER JOIN

```
SELECT e.employee_id, d.department_name, d.location_id
FROM employees e FULL OUTER JOIN departments d
ON e.department_id = d.department_id;
```

Set 14

Create table employees and department and solve the following queries

Name	Null?	Type

EMPLOYEE_ID	NOT NULL	NUMBER(6)
FIRST_NAME		VARCHAR2(20)
LAST_NAME	NOT NULL	VARCHAR2(25)
EMAIL	NOT NULL	VARCHAR2(25)
PHONE_NUMBER		VARCHAR2(20)
HIRE_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2(10)
SALARY		NUMBER(8,2)
COMMISSION_PCT		NUMBER(2,2)
MANAGER_ID		NUMBER(6)
DEPARTMENT_ID		NUMBER(4)

Name	Null?	Type

DEPARTMENT_ID	NOT NULL	NUMBER(4)
DEPARTMENT_NAME	NOT NULL	VARCHAR2(30)
MANAGER_ID		NUMBER(6)
LOCATION_ID		NUMBER(4)

1. Write a query to display the JOB,MGR,SAL,COMM,DNAME of employees whose JOB is SALESMAN.

2. The HR department needs a report of employees in Pune. Display the ENAME,job,DEPTNO and DNAME for all employees who work in Pune.

3. Display the Ename,Dname of all the employees,Also display those department name which do not have any employees working
4. Display the Ename,Dname of all the employees,Also display those departemnt name which do not have any employees working
5. Display the EMPNO,ENAME,LOC of those employees who are working as CLERK

1. SALESMAN report

```
SELECT job_id, manager_id, salary, commission_pct, department_name
FROM employees e, departments d
WHERE e.department_id = d.department_id
AND job_id = 'SALESMAN';
```

2. Employees working in Pune

```
SELECT first_name || ' ' || last_name ename, job_id, e.department_id, department_name
FROM employees e, departments d
WHERE e.department_id = d.department_id
AND location_id = 'Pune';
```

3. Employees + departments without employees (LEFT JOIN)

```
SELECT first_name || ' ' || last_name ename, department_name
FROM employees e RIGHT JOIN departments d
ON e.department_id = d.department_id;
```

4. Same as Q3 (corrected again)

```
SELECT first_name || ' ' || last_name ename, department_name
FROM employees e RIGHT JOIN departments d
ON e.department_id = d.department_id;
```

5. CLERK employees with location

```
SELECT employee_id, first_name || ' ' || last_name ename, location_id
FROM employees e, departments d
WHERE e.department_id = d.department_id
AND job_id = 'CLERK';
```

Set 15

Write a program that will prompt for a EMPNO,ENAME,SAL.If EMPNO does not exists in EMP table then INSERT the EMPNO,ENAME,SAL into the EMP table else UPDATE NAME,SAL for that EMPNO.

```
CREATE TABLE EMP (
EMPNO NUMBER PRIMARY KEY,
ENAME VARCHAR2(30),
SAL NUMBER(10,2)
);
```

```
INSERT INTO EMP VALUES (101, 'Amit', 25000);
INSERT INTO EMP VALUES (102, 'Neha', 30000);
```

```
COMMIT;
```

```

SET SERVEROUTPUT ON;

DECLARE
    v_empno EMP.EMPNO%TYPE := &EMPNO;
    v_ename EMP.ENAME%TYPE := '&ENAME';
    v_sal EMP.SAL%TYPE := &SAL;

    v_count NUMBER;

BEGIN

    SELECT COUNT(*)
    INTO v_count
    FROM EMP
    WHERE EMPNO = v_empno;

    IF v_count = 0 THEN

        INSERT INTO EMP
        VALUES (v_empno, v_ename, v_sal);

        DBMS_OUTPUT.PUT_LINE('Record Inserted Successfully');

    ELSE

        UPDATE EMP
        SET ENAME = v_ename,
            SAL = v_sal
        WHERE EMPNO = v_empno;

        DBMS_OUTPUT.PUT_LINE('Record Updated Successfully');

    END IF;

    COMMIT;

END;
/

```

Set 16

Write a program that will prompt for number and display whether the number is ODD or EVEN number. Add user defined exception to check the number is negative.

```

SET SERVEROUTPUT ON;

DECLARE
    num NUMBER := &num;

    negative_number EXCEPTION;

BEGIN

    /* Check Negative Number */

    IF num < 0 THEN

```

```
        RAISE negative_number;
    END IF;
```

```
/* Check Odd or Even */
```

```
IF MOD(num, 2) = 0 THEN
    DBMS_OUTPUT.PUT_LINE('Number is EVEN');
ELSE
    DBMS_OUTPUT.PUT_LINE('Number is ODD');
END IF;
```

```
EXCEPTION
```

```
    WHEN negative_number THEN
        DBMS_OUTPUT.PUT_LINE('Negative number is not allowed');
```

```
END;
```

```
/
```

Set 17

Write program that prompts for EMPNO and update his salary based on the below specification:

If the employee belongs to DEPTNO 10 then update his salary by 10%.

If the employee belongs to DEPTNO 20 then update his salary by 15%.

If the employee belongs to other DEPTNO then update his salary with SALARY+COMM.

```
CREATE TABLE EMP (
    EMPNO NUMBER PRIMARY KEY,
    ENAME VARCHAR2(30),
    SAL NUMBER(10,2),
    COMM NUMBER(10,2),
    DEPTNO NUMBER
);
```

```
INSERT INTO EMP VALUES (101, 'Amit', 30000, 2000, 10);
INSERT INTO EMP VALUES (102, 'Neha', 40000, 3000, 20);
INSERT INTO EMP VALUES (103, 'Rohit', 35000, 2500, 30);
```

```
COMMIT;
```

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    v_empno EMP.EMPNO%TYPE := &EMPNO;
```

```
    v_sal EMP.SAL%TYPE;
```

```
    v_comm EMP.COMM%TYPE;
```

```
    v_deptno EMP.DEPTNO%TYPE;
```

```
BEGIN
```

```
    SELECT SAL, COMM, DEPTNO
    INTO v_sal, v_comm, v_deptno
```



```

FROM EMP
WHERE EMPNO = v_empno;

IF v_deptno = 10 THEN

    v_sal := v_sal + (v_sal * 0.10);

ELSIF v_deptno = 20 THEN

    v_sal := v_sal + (v_sal * 0.15);

ELSE

    v_sal := v_sal + v_comm;

END IF;

UPDATE EMP
SET SAL = v_sal
WHERE EMPNO = v_empno;

DBMS_OUTPUT.PUT_LINE('Salary Updated Successfully');
DBMS_OUTPUT.PUT_LINE('Updated Salary = ' || v_sal);

COMMIT;

EXCEPTION

    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee Number Not Found');

END;
/

```

Set 18

Unnamed PL/SQL code block: Use of Control structure and Exception handling is mandatory. Write a PL/SQL block of code for the following requirements:-Consider table Stud(Roll, Att,Status) Write a PL/SQL block for following requirement and handle the exceptions. Roll no. of student will be entered by user. Attendance of roll no. entered by user will be checked in Stud table. If attendance is less than 75% then display the message “Term not granted” and set the status in stud table as “D”. Otherwise display message “Term granted” and set the status in stud table as “ND”

```

CREATE TABLE Stud (
    Roll NUMBER PRIMARY KEY,
    Att NUMBER(5,2),
    Status VARCHAR2(5)
);

INSERT INTO Stud VALUES (1, 80, NULL);
INSERT INTO Stud VALUES (2, 65, NULL);
INSERT INTO Stud VALUES (3, 72, NULL);

```

```
INSERT INTO Stud VALUES (4, 90, NULL);

COMMIT;

SET SERVEROUTPUT ON;

DECLARE

    v_rol Stud.Roll%TYPE := &ROLLNO;
    v_att Stud.Att%TYPE;

BEGIN

    SELECT Att
    INTO v_att
    FROM Stud
    WHERE Roll = v_rol;

    IF v_att < 75 THEN

        UPDATE Stud
        SET Status = 'D'
        WHERE Roll = v_rol;

        DBMS_OUTPUT.PUT_LINE('Term not granted');

    ELSE

        UPDATE Stud
        SET Status = 'ND'
        WHERE Roll = v_rol;

        DBMS_OUTPUT.PUT_LINE('Term granted');

    END IF;

    COMMIT;

EXCEPTION

    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Roll Number Not Found');

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error Occurred');

END;
/
```

Set 19

Unnamed PL/SQL code block: Use of Control structure and Exception handling is mandatory.

Write a PL/SQL block of code for the following requirements:-

Schema:

1. Borrower(Rollin, Name, DateofIssue, NameofBook, Status)

2. Fine(Roll_no,Date,Amt)

Accept roll_no & name of book from user.

Check the number of days (from date of issue), if days are between 15 to 30 then fine amount will be Rs 5per day.

If no. of days>30, per day fine will be Rs 50 per day & for days less than 30, Rs. 5 per day.

After submitting the book, status will change from I to R.

If condition of fine is true, then details will be stored into fine table.

```
CREATE TABLE Borrower (  
    Rollin NUMBER,  
    Name VARCHAR2(30),  
    DateofIssue DATE,  
    NameofBook VARCHAR2(50),  
    Status CHAR(1)  
);  
  
CREATE TABLE Fine (  
    Roll_no NUMBER,  
    Date1 DATE,  
    Amt NUMBER(10,2)  
);  
  
INSERT INTO Borrower  
VALUES (1, 'Amit',  
SYSDATE - 20,  
'DBMS', 'I');  
  
INSERT INTO Borrower  
VALUES (2, 'Neha',  
SYSDATE - 35,  
'CN', 'I');  
  
INSERT INTO Borrower  
VALUES (3, 'Rohit',  
SYSDATE - 10,  
'JAVA', 'I');  
  
COMMIT;  
  
SET SERVEROUTPUT ON;  
  
DECLARE  
  
    v_rol Borrower.Rollin%TYPE := &ROLLNO;  
    v_book Borrower.NameofBook%TYPE := '&BOOKNAME';  
  
    v_issue_date DATE;  
    v_days NUMBER;  
    v_fine NUMBER := 0;  
  
BEGIN
```

```
SELECT DateofIssue  
INTO v_issue_date  
FROM Borrower  
WHERE Rollin = v_roll  
AND NameofBook = v_book;
```

```
v_days := SYSDATE - v_issue_date;
```

```
IF v_days BETWEEN 15 AND 30 THEN
```

```
    v_fine := v_days * 5;
```

```
ELSIF v_days > 30 THEN
```

```
    v_fine := (30 * 5) + ((v_days - 30) * 50);
```

```
ELSE
```

```
    v_fine := 0;
```

```
END IF;
```

```
IF v_fine > 0 THEN
```

```
    INSERT INTO Fine  
    VALUES (v_roll, SYSDATE, v_fine);
```

```
END IF;
```

```
UPDATE Borrower  
SET Status = 'R'  
WHERE Rollin = v_roll  
AND NameofBook = v_book;
```

```
DBMS_OUTPUT.PUT_LINE('Book Returned Successfully');  
DBMS_OUTPUT.PUT_LINE('Total Fine = ' || v_fine);
```

```
COMMIT;
```

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN  
        DBMS_OUTPUT.PUT_LINE('Record Not Found');
```

```
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('Error Occurred');
```

```
END;
```

```
/
```

Set 20

Create following tables with primary and foreign key and solve the queries given below

```
student(S_ID,name,dept_name,tot_cred)
instructor(T_ID,name,dept_name,salary)
course(course_id,title,dept_name,credits)
```

- i. Find the average salary of instructor in those departments where the average salary is more than Rs. 42000/-.
- ii. Increase the salary of each instructor in the computer department by 10%.
- iii. Find the names of instructors whose names are neither 'Amol' nor 'Amit'.
- iv. Find the names of student which contains 'am' as its substring.
- v. Find the name of students from computer department that "DBMS" courses they take.

Create Tables

```
CREATE TABLE student (
  s_id NUMBER PRIMARY KEY,
  name VARCHAR2(30),
  dept_name VARCHAR2(30),
  tot_cred NUMBER
);
```

```
CREATE TABLE instructor (
  t_id NUMBER PRIMARY KEY,
  name VARCHAR2(30),
  dept_name VARCHAR2(30),
  salary NUMBER
);
```

```
CREATE TABLE course (
  course_id NUMBER PRIMARY KEY,
  title VARCHAR2(50),
  dept_name VARCHAR2(30),
  credits NUMBER
);
```

Insert Sample Values

Student

```
INSERT INTO student VALUES (1, 'Aman', 'Computer', 90);
INSERT INTO student VALUES (2, 'Ravi', 'IT', 85);
INSERT INTO student VALUES (3, 'Amita', 'Computer', 95);
INSERT INTO student VALUES (4, 'Soham', 'Mechanical', 80);
```

Instructor

```
INSERT INTO instructor VALUES (101, 'Amol', 'Computer', 50000);
INSERT INTO instructor VALUES (102, 'Amit', 'IT', 40000);
INSERT INTO instructor VALUES (103, 'Raj', 'Computer', 45000);
INSERT INTO instructor VALUES (104, 'Neha', 'Electronics', 60000);
```

Course

```
INSERT INTO course VALUES (201, 'DBMS', 'Computer', 4);
INSERT INTO course VALUES (202, 'OS', 'Computer', 4);
INSERT INTO course VALUES (203, 'Networking', 'IT', 3);
```

Queries

i. Avg salary > 42000

```
SELECT dept_name, AVG(salary)
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) > 42000;
```

ii. Increase salary (Computer dept)

```
UPDATE instructor
SET salary = salary * 1.10
WHERE dept_name = 'Computer';
```

iii. Names not Amol or Amit

```
SELECT name
FROM instructor
WHERE name NOT IN ('Amol', 'Amit');
```

iv. Names containing 'am'

```
SELECT name
FROM student
WHERE name LIKE '%am%';
```

v. Computer students taking DBMS

```
SELECT s.name
FROM student s, course c
WHERE s.dept_name = 'Computer'
AND c.title = 'DBMS';
```

Set 21

Write a program that displays the Top N salaries using simple Explicit Cursors.

```
/*-----
CREATE EMP TABLE
-----*/
```

```
CREATE TABLE EMP (
    EMPNO NUMBER PRIMARY KEY,
    ENAME VARCHAR2(30),
    SAL NUMBER(10,2)
);
```

```
/*-----
INSERT SAMPLE RECORDS
-----*/
```

```
INSERT INTO EMP VALUES (101, 'Amit', 25000);
INSERT INTO EMP VALUES (102, 'Neha', 45000);
INSERT INTO EMP VALUES (103, 'Rohit', 35000);
INSERT INTO EMP VALUES (104, 'Seema', 55000);
INSERT INTO EMP VALUES (105, 'Karan', 50000);
```

```
COMMIT;
```

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    v_n NUMBER := &N;
```

```
    v_empno EMP.EMPNO%TYPE;
```

```
    v_ename EMP.ENAME%TYPE;
```

```
    v_sal EMP.SAL%TYPE;
```

```
    counter NUMBER := 0;
```

```
    CURSOR c1 IS
```

```
    SELECT EMPNO, ENAME, SAL
```

```
    FROM EMP
```

```
    ORDER BY SAL DESC;
```

```
BEGIN
```

```
    OPEN c1;
```

```
    LOOP
```

```
        FETCH c1 INTO v_empno, v_ename, v_sal;
```

```
        EXIT WHEN c1%NOTFOUND OR counter = v_n;
```

```
        DBMS_OUTPUT.PUT_LINE
```

```
        (
```

```
            'EMPNO : ' || v_empno ||
```

```
            ' ENAME : ' || v_ename ||
```

```
            ' SALARY : ' || v_sal
```

```
        );
```

```
        counter := counter + 1;
```

```
    END LOOP;
```

```
    CLOSE c1;
```

```
END;
```

```
/
```

Set 22

PL/SQL Stored Procedure.

Write a Stored Procedure namely proc_Grade for the categorization of student. If marks scored by students in examination is ≤ 1500 and marks ≥ 990 then student will be placed in distinction category if marks scored are between 989 and 900 category is first class, if marks 899 and 825 category is Higher Second Class. Write a PL/SQL block for using procedure created with above requirement. Stud_Marks(name, total_marks) Result(Roll, Name, Class).

```
CREATE TABLE Stud_Marks (  
    Roll NUMBER PRIMARY KEY,  
    Name VARCHAR2(30),  
    Total_Marks NUMBER  
);
```

```
CREATE TABLE Result (  
    Roll NUMBER,  
    Name VARCHAR2(30),  
    Class VARCHAR2(30)  
);
```

```
INSERT INTO Stud_Marks  
VALUES (1, 'Amit', 1200);
```

```
INSERT INTO Stud_Marks  
VALUES (2, 'Neha', 950);
```

```
INSERT INTO Stud_Marks  
VALUES (3, 'Rohit', 850);
```

```
INSERT INTO Stud_Marks  
VALUES (4, 'Seema', 700);
```

```
COMMIT;
```

```
CREATE OR REPLACE PROCEDURE proc_Grade  
(  
    p_roll IN NUMBER,  
    p_name IN VARCHAR2,  
    p_marks IN NUMBER  
)
```

```
IS
```

```
    v_class VARCHAR2(30);
```

```
BEGIN
```

```
    IF p_marks BETWEEN 990 AND 1500 THEN
```

```
        v_class := 'Distinction';
```

```
    ELSIF p_marks BETWEEN 900 AND 989 THEN
```

```
        v_class := 'First Class';
```



```
ELSIF p_marks BETWEEN 825 AND 899 THEN
```

```
    v_class := 'Higher Second Class';
```

```
ELSE
```

```
    v_class := 'Fail';
```

```
END IF;
```

```
/* Insert Result */
```

```
INSERT INTO Result
```

```
VALUES (p_roll, p_name, v_class);
```

```
END;
```

```
/
```

```
BEGIN
```

```
    FOR rec IN
```

```
    (
```

```
        SELECT Roll, Name, Total_Marks
```

```
        FROM Stud_Marks
```

```
    )
```

```
    LOOP
```

```
        proc_Grade
```

```
        (
```

```
            rec.Roll,
```

```
            rec.Name,
```

```
            rec.Total_Marks
```

```
        );
```

```
    END LOOP;
```

```
    COMMIT;
```

```
END;
```

```
/
```

```
SELECT * FROM Result;
```

Set 23

Create below tables, insert 5 records each and solve the following queries

Name	Null?	Type

EMPLOYEE_ID	NOT NULL	NUMBER(6)
FIRST_NAME		VARCHAR2(20)
LAST_NAME	NOT NULL	VARCHAR2(25)
EMAIL	NOT NULL	VARCHAR2(25)
PHONE_NUMBER		VARCHAR2(20)
HIRE_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2(10)
SALARY		NUMBER(8,2)
COMMISSION_PCT		NUMBER(2,2)
MANAGER_ID		NUMBER(6)
DEPARTMENT_ID		NUMBER(4)

SQL> desc departments;

Name	Null?	Type

DEPARTMENT_ID	NOT NULL	NUMBER(4)
DEPARTMENT_NAME	NOT NULL	VARCHAR2(30)
MANAGER_ID		NUMBER(6)
LOCATION_ID		NUMBER(4)

1. List employees whose salary is greater than the salary of ALLEN.
2. Find employees whose job is the same as that of MARTIN.
3. Display employees who are not working in the department of KING.
4. Find employees who earn more than the salary of BLAKE.
5. Display employees hired after JONES.
6. Find employees whose department number is the same as CLARK.

```
CREATE TABLE employees (  
    employee_id NUMBER(6) PRIMARY KEY,  
    first_name VARCHAR2(20),  
    last_name VARCHAR2(25) NOT NULL,  
    email VARCHAR2(25) NOT NULL,  
    phone_number VARCHAR2(20),  
    hire_date DATE NOT NULL,  
    job_id VARCHAR2(10) NOT NULL,  
    salary NUMBER(8,2),  
    commission_pct NUMBER(2,2),  
    manager_id NUMBER(6),  
    department_id NUMBER(4)  
);  
  
CREATE TABLE departments (  
    department_id NUMBER(4) PRIMARY KEY,  
    department_name VARCHAR2(30) NOT NULL,  
    manager_id NUMBER(6),  
    location_id NUMBER(4)  
);
```

Insert 5 Records (Employees)

```
INSERT INTO employees VALUES (1,'John','ALLEN','a@x.com','111',TO_DATE('2020-01-01','YYYY-MM-DD'),'CLERK',3000,NULL,101,10);
INSERT INTO employees VALUES (2,'Mike','MARTIN','b@x.com','222',TO_DATE('2021-02-01','YYYY-MM-DD'),'MANAGER',5000,NULL,102,20);
INSERT INTO employees VALUES (3,'Sara','JONES','c@x.com','333',TO_DATE('2022-03-01','YYYY-MM-DD'),'CLERK',4000,NULL,103,30);
INSERT INTO employees VALUES (4,'David','BLAKE','d@x.com','444',TO_DATE('2023-04-01','YYYY-MM-DD'),'ANALYST',6000,NULL,104,40);
INSERT INTO employees VALUES (5,'Emma','CLARK','e@x.com','555',TO_DATE('2024-05-01','YYYY-MM-DD'),'CLERK',7000,NULL,105,50);
```

Insert 5 Records (Departments)

```
INSERT INTO departments VALUES (10,'HR',101,1000);
INSERT INTO departments VALUES (20,'IT',102,2000);
INSERT INTO departments VALUES (30,'SALES',103,3000);
INSERT INTO departments VALUES (40,'FINANCE',104,4000);
INSERT INTO departments VALUES (50,'ADMIN',105,5000);
```

Queries

1. Salary greater than ALLEN

```
SELECT *
FROM employees
WHERE salary > (SELECT salary FROM employees WHERE last_name = 'ALLEN');
```

2. Same job as MARTIN

```
SELECT *
FROM employees
WHERE job_id = (SELECT job_id FROM employees WHERE last_name = 'MARTIN');
```

3. Not working in KING's department

```
SELECT *
FROM employees
WHERE department_id <> (SELECT department_id FROM employees WHERE last_name = 'KING');
```

4. Salary more than BLAKE

```
SELECT *
FROM employees
WHERE salary > (SELECT salary FROM employees WHERE last_name = 'BLAKE');
```

5. Hired after JONES

```
SELECT *
FROM employees
WHERE hire_date > (SELECT hire_date FROM employees WHERE last_name = 'JONES');
```

6. Same department as CLARK

```
SELECT *
FROM employees
WHERE department_id = (SELECT department_id FROM employees WHERE last_name = 'CLARK');
```

Set 24

Create below tables employee and department, insert 5 records each and solve the following queries

Name	Null?	Type

EMPLOYEE_ID	NOT NULL	NUMBER(6)
FIRST_NAME		VARCHAR2(20)
LAST_NAME	NOT NULL	VARCHAR2(25)
EMAIL	NOT NULL	VARCHAR2(25)
PHONE_NUMBER		VARCHAR2(20)
HIRE_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2(10)
SALARY		NUMBER(8,2)
COMMISSION_PCT		NUMBER(2,2)
MANAGER_ID		NUMBER(6)
DEPARTMENT_ID		NUMBER(4)

Name	Null?	Type

DEPARTMENT_ID	NOT NULL	NUMBER(4)
DEPARTMENT_NAME	NOT NULL	VARCHAR2(30)
MANAGER_ID		NUMBER(6)
LOCATION_ID		NUMBER(4)

1. Find employees whose salary is greater than ANY employee in department 30.
2. Display employees whose salary is greater than ALL employees in department 20.
3. List employees working in departments where the job is MANAGER.
4. Find employees whose salary is greater than the average salary of their own department.
5. Display employees who earn the highest salary in their department.
6. Find employees who have the same job as at least one employee in department 20.

Create Tables

```
CREATE TABLE employee (  
  employee_id NUMBER(6) PRIMARY KEY,  
  first_name VARCHAR2(20),  
  last_name VARCHAR2(25) NOT NULL,  
  email VARCHAR2(25) NOT NULL,  
  phone_number VARCHAR2(20),  
  hire_date DATE NOT NULL,  
  job_id VARCHAR2(10) NOT NULL,  
  salary NUMBER(8,2),  
  commission_pct NUMBER(2,2),  
  manager_id NUMBER(6),  
  department_id NUMBER(4)  
);  
  
CREATE TABLE department (  
  department_id NUMBER(4) PRIMARY KEY,  
  department_name VARCHAR2(30) NOT NULL,  
  manager_id NUMBER(6),  
  location_id NUMBER(4)  
);
```

Insert 5 Records (Employee)

```
INSERT INTO employee VALUES (1,'A','ALLEN','a@x.com','111',TO_DATE('2021-01-01','YYYY-MM-DD'),'CLERK',3000,NULL,101,10);
```

```
INSERT INTO employee VALUES (2,'B','MARTIN','b@x.com','222',TO_DATE('2021-02-01','YYYY-MM-DD'),'MANAGER',5000,NULL,102,20);
INSERT INTO employee VALUES (3,'C','JONES','c@x.com','333',TO_DATE('2021-03-01','YYYY-MM-DD'),'CLERK',7000,NULL,103,30);
INSERT INTO employee VALUES (4,'D','BLAKE','d@x.com','444',TO_DATE('2021-04-01','YYYY-MM-DD'),'MANAGER',9000,NULL,104,20);
INSERT INTO employee VALUES (5,'E','CLARK','e@x.com','555',TO_DATE('2021-05-01','YYYY-MM-DD'),'CLERK',6000,NULL,105,30);
```

Insert 5 Records (Department)

```
INSERT INTO department VALUES (10,'HR',101,1000);
INSERT INTO department VALUES (20,'IT',102,2000);
INSERT INTO department VALUES (30,'SALES',103,3000);
INSERT INTO department VALUES (40,'FINANCE',104,4000);
INSERT INTO department VALUES (50,'ADMIN',105,5000);
```

Queries

1. Salary greater than ANY in dept 30

```
SELECT *
FROM employee
WHERE salary > ANY (SELECT salary FROM employee WHERE department_id = 30);
```

2. Salary greater than ALL in dept 20

```
SELECT *
FROM employee
WHERE salary > ALL (SELECT salary FROM employee WHERE department_id = 20);
```

3. Employees in departments where job is MANAGER

```
SELECT *
FROM employee
WHERE department_id IN (
    SELECT department_id FROM employee WHERE job_id = 'MANAGER'
);
```

4. Salary greater than department average

```
SELECT *
FROM employee e
WHERE salary > (
    SELECT AVG(salary)
    FROM employee
    WHERE department_id = e.department_id
);
```

5. Highest salary in each department

```
SELECT *
FROM employee e
WHERE salary = (
    SELECT MAX(salary)
    FROM employee
    WHERE department_id = e.department_id
);
```

6. Same job as at least one employee in dept 20

```
SELECT *
FROM employee
WHERE job_id IN (
    SELECT job_id FROM employee WHERE department_id = 20
);
```

Set 25

Write a database trigger on Library table. The System should keep track of the records that are being updated or deleted. The old value of updated or deleted records should be added in Library_ Audit table. Frame the problem statement for writing Database Triggers of all types, in-line with above statement. The problem statement should clearly state the requirements.

1) CREATE LIBRARY TABLE

-----*/

```
CREATE TABLE Library (  
    Book_ID NUMBER PRIMARY KEY,  
    Book_Name VARCHAR2(50),  
    Author VARCHAR2(30),  
    Price NUMBER(10,2)  
);
```

/*-----

2) CREATE LIBRARY_AUDIT TABLE

-----*/

```
CREATE TABLE Library_Audit (  
    Book_ID NUMBER,  
    Book_Name VARCHAR2(50),  
    Author VARCHAR2(30),  
    Price NUMBER(10,2),  
    Operation_Type VARCHAR2(20),  
    Operation_Date DATE  
);
```

/*-----

3) INSERT SAMPLE RECORDS

-----*/

```
INSERT INTO Library  
VALUES (101, 'DBMS', 'Korth', 500);
```

```
INSERT INTO Library  
VALUES (102, 'JAVA', 'James', 450);
```

```
INSERT INTO Library  
VALUES (103, 'CN', 'Tanenbaum', 600);
```

```
COMMIT;
```

/*-----

4) BEFORE UPDATE TRIGGER

STORE OLD VALUES

-----*/

```
CREATE OR REPLACE TRIGGER trg_before_update  
BEFORE UPDATE  
ON Library
```

FOR EACH ROW

BEGIN

```
INSERT INTO Library_Audit
VALUES (
    :OLD.Book_ID,
    :OLD.Book_Name,
    :OLD.Author,
    :OLD.Price,
    'BEFORE UPDATE',
    SYSDATE
);
```

```
END;
/
```

```
/*-----
5) AFTER UPDATE TRIGGER
-----*/
```

```
CREATE OR REPLACE TRIGGER trg_after_update
AFTER UPDATE
ON Library
FOR EACH ROW
```

BEGIN

```
INSERT INTO Library_Audit
VALUES (
    :OLD.Book_ID,
    :OLD.Book_Name,
    :OLD.Author,
    :OLD.Price,
    'AFTER UPDATE',
    SYSDATE
);
```

```
END;
/
```

```
/*-----
6) BEFORE DELETE TRIGGER
-----*/
```

```
CREATE OR REPLACE TRIGGER trg_before_delete
BEFORE DELETE
ON Library
FOR EACH ROW
```

BEGIN

```
INSERT INTO Library_Audit
VALUES (
    :OLD.Book_ID,
```

```

        :OLD.Book_Name,
        :OLD.Author,
        :OLD.Price,
        'BEFORE DELETE',
        SYSDATE
    );

END;
/

/*-----
7) AFTER DELETE TRIGGER
-----*/

CREATE OR REPLACE TRIGGER trg_after_delete
AFTER DELETE
ON Library
FOR EACH ROW

BEGIN

    INSERT INTO Library_Audit
    VALUES (
        :OLD.Book_ID,
        :OLD.Book_Name,
        :OLD.Author,
        :OLD.Price,
        'AFTER DELETE',
        SYSDATE
    );

END;
/

/*-----
8) TEST UPDATE TRIGGER
-----*/

UPDATE Library
SET Price = 700
WHERE Book_ID = 101;

COMMIT;

/*-----
9) TEST DELETE TRIGGER
-----*/

DELETE FROM Library
WHERE Book_ID = 102;

COMMIT;

```



```
/*-----  
10) DISPLAY AUDIT TABLE  
-----*/
```

```
SELECT * FROM Library_Audit;
```

Set 26

Collection "orderinfo" which contains the documents given as below

```
{  
  cust_id:123  
  cust_name:"abc",  
  status:"A",  
  price:250  
}
```

- 1) find the average price for each customers having status 'A'
- 2) Display the status of the customers whose amount/price lie between 100 and 1000
- 3) Display the customers information without "_id" .
- 4) create a simple index on onderinfo collection and fire the queries.

use orderdb

Insert Sample Documents (if required)

```
db.orderinfo.insertMany([  
  { cust_id: 123, cust_name: "abc", status: "A", price: 250 },  
  { cust_id: 124, cust_name: "def", status: "A", price: 600 },  
  { cust_id: 125, cust_name: "ghi", status: "B", price: 90 },  
  { cust_id: 123, cust_name: "abc", status: "A", price: 450 },  
  { cust_id: 126, cust_name: "jkl", status: "A", price: 1200 }  
])
```

1. Average price for each customer (status = 'A')

```
db.orderinfo.aggregate([  
  { $match: { status: "A" } },  
  { $group: { _id: "$cust_id", avg_price: { $avg: "$price" } } }  
])
```

2. Price between 100 and 1000

```
db.orderinfo.find({  
  price: { $gte: 100, $lte: 1000 }  
})
```

3. Display all data without _id

```
db.orderinfo.find({}, { _id: 0 })
```

4. Create simple index on price field

```
db.orderinfo.createIndex({ price: 1 })
```

Set 27

Collection "movies" which contains the documents given as below(Perform on Mongo Terminal)

```
{
  name: "Movie1",
  type: "action",
  budget: 1000000
  producer: {
    name: "producer1",
    address: "PUNE"
  }
}
```

- i. Find the name of the movie having budget greater than 1,00,000.
- ii. Find the name of producer who lives in Pune
- iii. Update the type of movie "action" to "horror"
- iv. Find all the documents produced by name "producer1" with their address

Use Database

use moviedb

Insert Sample Documents (if required)

```
db.movies.insertMany([
  {
    name: "Movie1",
    type: "action",
    budget: 1000000,
    producer: { name: "producer1", address: "PUNE" }
  },
  {
    name: "Movie2",
    type: "action",
    budget: 50000,
    producer: { name: "producer2", address: "MUMBAI" }
  },
  {
    name: "Movie3",
    type: "drama",
    budget: 200000,
    producer: { name: "producer1", address: "PUNE" }
  }
])
```

i. Movies with budget > 1,00,000

```
db.movies.find(
  { budget: { $gt: 100000 } },
  { name: 1, _id: 0 }
)
```

ii. Producer name who lives in Pune

```
db.movies.find(
  { "producer.address": "PUNE" },
  { "producer.name": 1, _id: 0 }
)
```

iii. Update type from action to horror

```
db.movies.updateMany(
  { type: "action" },
  { $set: { type: "horror" } }
)
```

iv. Producer1 details with address

```
db.movies.find(
  { "producer.name": "producer1" },
  { producer: 1, _id: 0 }
)
```

Set 28

Consider following structure for MongoDB collection and write a query for following requirements in MongoDB

Teachers (Tname,dno,Experience,Salary,Date_of_Joining)

Department (Dno,Dname)

Students(Sname,Roll_No,Class)

1. Write a query to create above collection insert some sample documents.
2. Find the information about all teachers of Dno=2 and having salary greater than or equal to 10,000/-
3. Find the student information having Roll_no=2 or Sname='xyz'
4. Update student name whose Roll_No=5
5. Delete all student whose Class is 'FE'
6. Apply index on Students Collection

1. Create collections + insert sample documents

```
db.Teachers.insertMany([
  { Tname: "Amit", dno: 1, Experience: 5, Salary: 12000, Date_of_Joining: ISODate("2020-01-10") },
  { Tname: "Ravi", dno: 2, Experience: 8, Salary: 15000, Date_of_Joining: ISODate("2018-05-15") },
  { Tname: "Neha", dno: 2, Experience: 3, Salary: 9000, Date_of_Joining: ISODate("2022-07-01") }
])
db.Department.insertMany([
  { Dno: 1, Dname: "Computer" },
  { Dno: 2, Dname: "IT" }
])
db.Students.insertMany([
  { Sname: "xyz", Roll_No: 2, Class: "FE" },
  { Sname: "abc", Roll_No: 5, Class: "SE" },
  { Sname: "pqr", Roll_No: 3, Class: "FE" }
])
```

2. Teachers of Dno=2 and Salary >= 10000

```
db.Teachers.find({
  dno: 2,
  Salary: { $gte: 10000 }
})
```

3. Student info Roll_No=2 OR Sname='xyz'

```
db.Students.find({
  $or: [
    { Roll_No: 2 },
    { Sname: "xyz" }
  ]
})
```

4. Update student name where Roll_No=5

```
db.Students.updateOne(
  { Roll_No: 5 },
  { $set: { Sname: "updated_name" } }
)
```

5. Delete students whose Class = 'FE'

```
db.Students.deleteMany({ Class: "FE" })
```

6. Create index on Students collection

```
db.Students.createIndex({ Roll_No: 1 })
```

Set 29

Perform aggregation and Indexing using mongodb on below database

1. Create a database department
2. Create a collection as teacher with fields as name , department ,experience and salary
3. Display the department wise average salary.
4. Display the no. Of employees working in each department.
5. Display the department wise minimum salary.
6. Apply index and drop index

1. Create Database

use department

2. Create Collection + Insert Sample Data

```
db.teacher.insertMany([
  { name: "Amit", department: "CSE", experience: 5, salary: 50000 },
  { name: "Ravi", department: "CSE", experience: 3, salary: 40000 },
  { name: "Neha", department: "IT", experience: 4, salary: 45000 },
  { name: "Raj", department: "IT", experience: 6, salary: 60000 },
  { name: "Pooja", department: "MECH", experience: 2, salary: 30000 }
])
```

3. Department wise average salary

```
db.teacher.aggregate([
  { $group: { _id: "$department", avg_salary: { $avg: "$salary" } } }
])
```

4. Number of employees in each department

```
db.teacher.aggregate([
  { $group: { _id: "$department", count: { $sum: 1 } } }
])
```

5. Department wise minimum salary

```
db.teacher.aggregate([
  { $group: { _id: "$department", min_salary: { $min: "$salary" } } }
])
```

6. Indexing + Drop Index

Create Index on department

```
db.teacher.createIndex({ department: 1 })
```

View indexes

```
db.teacher.getIndexes()
```

Drop index

```
db.teacher.dropIndex({ department: 1 })
```

Set 30

1. Create database product_detail
2. Show Database
3. Create collection product
4. Show collection
5. Insert below documents

```
{ "item": "pen", "price": 10, "quantity": 2 } { "item": "book", "price": 50, "quantity": 1 } { "item": "pen", "price": 10, "quantity": 5 } { "item": "pencil", "price": 5, "quantity": 10 }
```
6. Retrieve documents from collection.
7. Create index on item in ascending order
8. Show index
9. Create index on price in ascending and quantity as descending
10. Drop index on price

1. Create Database

use product_detail

2. Show Database

show dbs

3. Create Collection

```
db.createCollection("product")
```

4. Show Collections

show collections

5. Insert Documents

```
db.product.insertMany([
  { item: "pen", price: 10, quantity: 2 },
  { item: "book", price: 50, quantity: 1 },
  { item: "pen", price: 10, quantity: 5 },
  { item: "pencil", price: 5, quantity: 10 }
])
```

6. Retrieve Documents

```
db.product.find()
```

7. Create Index on item (ascending)

```
db.product.createIndex({ item: 1 })
```

8. Show Indexes

```
db.product.getIndexes()
```

9. Index on price (asc) and quantity (desc)

```
db.product.createIndex({ price: 1, quantity: -1 })
```

10. Drop index on price

```
db.product.dropIndex({ price: 1 })
```

Set 31

Collection "orderinfo" which contains the documents given as below

```
{
  cust_id:123
  cust_name:"abc",
  status:"A",
  price:250
}
```

- i. Display the name of the customer having the price between 250 and 450
- ii. Increment the price by 10 for cust_id: 123 and decrement the price by 5 for cust_id: 124
- iii. Remove any one of the field from the orderinfo collection.
- iv. Find the name of the customer whose status is either A or price is 250 or both.

Use Database

use orderdb

Insert Records (if required)

```
db.orderinfo.insertMany([
  { cust_id: 123, cust_name: "abc", status: "A", price: 250 },
  { cust_id: 124, cust_name: "def", status: "B", price: 400 },
  { cust_id: 125, cust_name: "ghi", status: "A", price: 300 },
  { cust_id: 126, cust_name: "jkl", status: "C", price: 450 }
])
```

i. Price between 250 and 450

```
db.orderinfo.find(
  { price: { $gte: 250, $lte: 450 } },
  { cust_name: 1, _id: 0 }
)
```

ii. Update price

```
db.orderinfo.updateOne(
  { cust_id: 123 },
  { $inc: { price: 10 } }
)
db.orderinfo.updateOne(
  { cust_id: 124 },
  { $inc: { price: -5 } }
)
```

iii. Remove field (example: status)

```
db.orderinfo.updateMany(
  {},
  { $unset: { status: "" } }
)
```

iv. status = A OR price = 250

```
db.orderinfo.find({
  $or: [
    { status: "A" },
    { price: 250 }
  ]
})
```

Set 32

Emp(emp_id,ename, street, city)

works(emp_id,c_id,ename, cname, sal)

Company(c_id,cname, city)

Manager(mgr_id, mgrname)

i. Modify the database so that a particular company (eg. ABC) now in Pune

ii. Give all managers of Mbank a 10% raise. If salary is >20,000, give only 3% raise.

iii. Find out the names of all the employees who works in 'Bosch' company in city Pune

iv. Delete all records in the works table for employees of a particular company (Eg, SBC Company) whose salary>50,000.

Create Tables

```
CREATE TABLE Emp (  
    emp_id NUMBER PRIMARY KEY,  
    ename VARCHAR2(30),  
    street VARCHAR2(30),  
    city VARCHAR2(30)  
);  
CREATE TABLE Company (  
    c_id NUMBER PRIMARY KEY,  
    cname VARCHAR2(30),  
    city VARCHAR2(30)  
);  
CREATE TABLE Works (  
    emp_id NUMBER,  
    c_id NUMBER,  
    ename VARCHAR2(30),  
    cname VARCHAR2(30),  
    sal NUMBER  
);  
CREATE TABLE Manager (  
    mgr_id NUMBER PRIMARY KEY,  
    mgrname VARCHAR2(30)  
);
```

Insert Records

```
INSERT INTO Emp VALUES (1,'Amit','Street1','Pune');  
INSERT INTO Emp VALUES (2,'Ravi','Street2','Mumbai');  
INSERT INTO Emp VALUES (3,'Neha','Street3','Pune');  
INSERT INTO Emp VALUES (4,'John','Street4','Delhi');  
INSERT INTO Company VALUES (101,'ABC','Mumbai');  
INSERT INTO Company VALUES (102,'Mbank','Pune');  
INSERT INTO Company VALUES (103,'Bosch','Pune');  
INSERT INTO Company VALUES (104,'SBC','Delhi');  
INSERT INTO Works VALUES (1,101,'Amit','ABC',40000);  
INSERT INTO Works VALUES (2,102,'Ravi','Mbank',25000);  
INSERT INTO Works VALUES (3,103,'Neha','Bosch',60000);  
INSERT INTO Works VALUES (4,104,'John','SBC',70000);  
INSERT INTO Manager VALUES (201,'Manager1');  
INSERT INTO Manager VALUES (202,'Manager2');
```

i. Move ABC company to Pune

```
UPDATE Company  
SET city = 'Pune'  
WHERE cname = 'ABC';
```

ii. Salary update for Mbank

```
UPDATE Works
SET sal = sal * 1.10
WHERE cname = 'Mbank' AND sal <= 20000;
UPDATE Works
SET sal = sal * 1.03
WHERE cname = 'Mbank' AND sal > 20000;
```

iii. Employees in Bosch (Pune)

```
SELECT e.ename
FROM Emp e, Works w, Company c
WHERE e.emp_id = w.emp_id
AND w.c_id = c.c_id
AND c.cname = 'Bosch'
AND c.city = 'Pune';
```

iv. Delete SBC employees with salary > 50000

```
DELETE FROM Works
WHERE cname = 'SBC'
AND sal > 50000;
```

Set 33

1. Create database Employee.
2. Create collection emp1 using createCollection method.
3. Insert 4 to 5 documents in emp1 collection.(eno,ename,address,sal)
4. display all documents.
5. Display all employess having salary greater than 5000
6. Display all employess having salary less than 15000
7. Display all employess having salary greater than 10000 and less than 20000.
8. Update sal of all employee with 10%
9. Delete employee having salary less than 5000.
10. Rename collection emp1 with employee1.
11. Display employee whose name start with n.
12. sort name in ascending order.
13. Create a new database with name Employee1.
14. Drop employee1 database.
15. Create collection emp1 using insert method.
16. Drop collection emp1.

1. Create Database

use Employee

2. Create Collection using createCollection

```
db.createCollection("emp1")
```

3. Insert Documents

```
db.emp1.insertMany([
  { eno: 1, ename: "Neha", address: "Pune", sal: 12000 },
  { eno: 2, ename: "Amit", address: "Mumbai", sal: 8000 },
  { eno: 3, ename: "Nikhil", address: "Nashik", sal: 4000 },
```



```
{ eno: 4, ename: "Ravi", address: "Delhi", sal: 20000 }  
])
```

4. Display all documents

```
db.emp1.find()
```

5. Salary > 5000

```
db.emp1.find({ sal: { $gt: 5000 } })
```

6. Salary < 15000

```
db.emp1.find({ sal: { $lt: 15000 } })
```

7. Salary between 10000 and 20000

```
db.emp1.find({ sal: { $gt: 10000, $lt: 20000 } })
```

8. Increase salary by 10%

```
db.emp1.updateMany(  
  {},  
  { $mul: { sal: 1.10 } }  
)
```

9. Delete salary < 5000

```
db.emp1.deleteMany({ sal: { $lt: 5000 } })
```

10. Rename collection emp1 to employee1

```
db.emp1.renameCollection("employee1")
```

11. Name starts with 'n'

```
db.employee1.find({ ename: /^n/i })
```

12. Sort name ascending

```
db.employee1.find().sort({ ename: 1 })
```

13. Create new database Employee1

```
use Employee1
```

14. Drop Employee1 database

```
db.dropDatabase()
```

15. Create collection using insert method

```
db.emp1.insertOne({ eno: 5, ename: "Sana", address: "Goa", sal: 15000 })
```

16. Drop collection emp1

```
db.emp1.drop()
```

Set 34

Employee(employee_name,street,city)
Works(employee_name,company_name,salary)
Company(company_name,city)
Manages(employee_name,manager_name)

1. Find the names of employees who work for First Bank Corporation.
2. Find the names and cities of residence of all employees who work for First Bank Corporation
3. Find the names, street addresses, and cities of residence of all employees who work for First Bank Corporation and earn more than \$10000.
4. Find all employees in the database who lives in the same cities as the companies for which they work.
5. Find all employees in the database who lives in the same cities and on the same streets as do their manager.
6. Find all employees in the database who do not work for First Bank Corporation
7. Give all employees of First Bank Corporation a 10% raise
8. Delete all tuples in the "Works" relation for employees of "Small Bank Corporation".

Create Tables

```
CREATE TABLE Employee (  
    employee_name VARCHAR2(30) PRIMARY KEY,  
    street VARCHAR2(30),  
    city VARCHAR2(30)  
);  
CREATE TABLE Works (  
    employee_name VARCHAR2(30),  
    company_name VARCHAR2(30),  
    salary NUMBER  
);  
CREATE TABLE Company (  
    company_name VARCHAR2(30) PRIMARY KEY,  
    city VARCHAR2(30)  
);  
CREATE TABLE Manages (  
    employee_name VARCHAR2(30),  
    manager_name VARCHAR2(30)  
);
```

Insert Values

Employee

```
INSERT INTO Employee VALUES ('Amit','MG Road','Pune');  
INSERT INTO Employee VALUES ('Ravi','Street1','Mumbai');  
INSERT INTO Employee VALUES ('Neha','Street2','Pune');  
INSERT INTO Employee VALUES ('John','Street3','Delhi');
```

Company

```
INSERT INTO Company VALUES ('First Bank Corporation','Pune');  
INSERT INTO Company VALUES ('Small Bank Corporation','Mumbai');  
INSERT INTO Company VALUES ('TechSoft','Delhi');
```

Works

```
INSERT INTO Works VALUES ('Amit','First Bank Corporation',12000);  
INSERT INTO Works VALUES ('Ravi','First Bank Corporation',9000);  
INSERT INTO Works VALUES ('Neha','Small Bank Corporation',15000);  
INSERT INTO Works VALUES ('John','TechSoft',8000);
```

Manages

```
INSERT INTO Manages VALUES ('Amit','Ravi');  
INSERT INTO Manages VALUES ('Neha','John');
```

Queries

1. Employees of First Bank Corporation

```
SELECT employee_name
```

```
FROM Works
WHERE company_name = 'First Bank Corporation';
```

2. Names and cities of employees

```
SELECT e.employee_name, e.city
FROM Employee e, Works w
WHERE e.employee_name = w.employee_name
AND w.company_name = 'First Bank Corporation';
```

3. Employees earning > 10000

```
SELECT e.employee_name, e.street, e.city
FROM Employee e, Works w
WHERE e.employee_name = w.employee_name
AND w.company_name = 'First Bank Corporation'
AND w.salary > 10000;
```

4. Same city as company

```
SELECT e.employee_name
FROM Employee e, Works w, Company c
WHERE e.employee_name = w.employee_name
AND w.company_name = c.company_name
AND e.city = c.city;
```

5. Same street & city as manager

```
SELECT e.employee_name
FROM Employee e, Manages m, Employee m2
WHERE e.employee_name = m.employee_name
AND m.manager_name = m2.employee_name
AND e.city = m2.city
AND e.street = m2.street;
```

6. Not working for First Bank Corporation

```
SELECT employee_name
FROM Works
WHERE company_name <> 'First Bank Corporation';
```

7. Give 10% raise

```
UPDATE Works
SET salary = salary * 1.10
WHERE company_name = 'First Bank Corporation';
```

8. Delete Small Bank Corporation employees

```
DELETE FROM Works
WHERE company_name = 'Small Bank Corporation';
```

Set 35

```
Employee(employee_name,street,city)
Works(employee_name,company_name,salary)
Company(company_name,city)
Manages(employee_name,manager_name)
```

- 1) Find all employees in the database who earn more than each employee of Small Bank Corporation
- 2) Assume that the company is may be located in several cities. Find all companies located in every city in which Small Bank Corporation is located.
- 3) Find all employees who earn more than the average salary of all employees of their companies.
- 4) Find the company that has the most employees.
- 5) Find the company that has the smallest payroll.
- 6) Find those companies whose employees earn a higher salary, on average, than the average salary at First Bank Corporation.
- 7) Modify the database so that "Jones" now lives in Newtown.

Create Tables

```
CREATE TABLE Employee (
  employee_name VARCHAR2(30) PRIMARY KEY,
  street VARCHAR2(30),
  city VARCHAR2(30)
);
CREATE TABLE Works (
  employee_name VARCHAR2(30),
  company_name VARCHAR2(30),
  salary NUMBER
);
CREATE TABLE Company (
  company_name VARCHAR2(30) PRIMARY KEY,
  city VARCHAR2(30)
);
CREATE TABLE Manages (
  employee_name VARCHAR2(30),
  manager_name VARCHAR2(30)
);
```

Insert Sample Data**Employee**

```
INSERT INTO Employee VALUES ('Jones','Street1','Pune');
INSERT INTO Employee VALUES ('Amit','Street2','Mumbai');
INSERT INTO Employee VALUES ('Ravi','Street3','Delhi');
INSERT INTO Employee VALUES ('Neha','Street4','Pune');
```

Company

```
INSERT INTO Company VALUES ('First Bank Corporation','Pune');
INSERT INTO Company VALUES ('Small Bank Corporation','Mumbai');
INSERT INTO Company VALUES ('TechSoft','Delhi');
```

Works

```
INSERT INTO Works VALUES ('Jones','First Bank Corporation',12000);
INSERT INTO Works VALUES ('Amit','Small Bank Corporation',8000);
INSERT INTO Works VALUES ('Ravi','Small Bank Corporation',9000);
INSERT INTO Works VALUES ('Neha','TechSoft',15000);
```

Manages

```
INSERT INTO Manages VALUES ('Jones','Amit');
INSERT INTO Manages VALUES ('Neha','Ravi');
```

Queries

1. Salary greater than all employees of Small Bank Corporation

```
SELECT employee_name
FROM Works
WHERE salary > ALL (
    SELECT salary
    FROM Works
    WHERE company_name = 'Small Bank Corporation'
);
```

2. Companies in every city of Small Bank Corporation

```
SELECT c1.company_name
FROM Company c1
WHERE NOT EXISTS (
    SELECT city
    FROM Company
    WHERE company_name = 'Small Bank Corporation'
    MINUS
    SELECT city
    FROM Company c2
    WHERE c2.company_name = c1.company_name
);
```

3. Salary greater than company average

```
SELECT w.employee_name
FROM Works w
WHERE salary > (
    SELECT AVG(salary)
    FROM Works
    WHERE company_name = w.company_name
);
```

4. Company with most employees

```
SELECT company_name
FROM Works
GROUP BY company_name
HAVING COUNT(*) = (
    SELECT MAX(COUNT(*))
    FROM Works
    GROUP BY company_name
);
```

5. Company with smallest payroll

```
SELECT company_name
FROM Works
GROUP BY company_name
HAVING SUM(salary) = (
    SELECT MIN(SUM(salary))
    FROM Works
    GROUP BY company_name
);
```

6. Companies with higher avg salary than First Bank Corporation

```
SELECT company_name
FROM Works
GROUP BY company_name
HAVING AVG(salary) > (
    SELECT AVG(salary)
    FROM Works
    WHERE company_name = 'First Bank Corporation'
);
```

```
FROM Works
WHERE company_name = 'First Bank Corporation'
);
```

7. Update Jones city to Newtown

```
UPDATE Employee
SET city = 'Newtown'
WHERE employee_name = 'Jones';
```